CSE107 — Lab #4

Random Number Generator on Arduino

Lab Date: 24/10/2025 — Due: 31/10/2025 08:00 am

Student: Name Surname: Zeynep Özge TOPTAŞ
Student ID: 240104004006
Section: 2
Instructor: Zeliha ERİM

1. Objectives

This labwork aimed to develop a number guessing game on the Arduino. The project defined variables (N, target, lowVal, highVal, guess) to hold the game range and search boundaries, and used constant integer pins (Led, Led2) for LED feedback. The setup() function initializes serial communication and sets LED pins as outputs. Crucially, it ensures enhanced randomness by setting the random seed using electrical noise from an unconnected pin via the randomSeed(analogRead(A0)) command. Following setup, the code uses some functions, readUnsignedLongFromSerial and readCharFromSerial, to securely obtain the initial inputs from the user: the maximum game limit (N) and the secret target number. The core loop() function contains the main game logic, which generates a random guess value within the current search range, checks if it matches the target, and if not, updates the search boundaries (lowVal or highVal) based on the user's input of 's' (smaller) or 'b' (bigger).

2. Environment

Fill in the following ones you used in the lab and give brief information for each one of them.

- **Operating System:** Linux
- **Board:** Arduino Uno
- **IDE:** Arduino IDE (Ubuntu)

3. Problem Solving

3.1. Think that the case you are facing with the statement that you want to show only once in the setup process appears multiple times in the terminal, what kind of solutions would you bring to solve the problem? Explain what your solutions change and how they become beneficial to the system.

Solution for 3.1: The issue of output appearing multiple times when intended for only one

execution in setup() is typically caused by the Serial Monitor quickly disconnecting and reconnecting after a board reset, causing setup() to execute more than once. To solve this, I added while (!Serial); after Serial.begin(). This critically ensures the program pauses until the Serial Monitor is fully connected, preventing initial output from being missed or duplicated upon synchronization.

3.2. On an Arduino board, if you obtain the same numbers when trying to generate random integers, what is the underlying cause of the problem, and are they hardware or software-related issues? Explain what sort of strategies you might follow to address the problem.

Solution for 3.2: The problem is that the Arduino's random() function isn't truly random it uses a formula that always produces the same number sequence if it starts with the same initial value (the seed). To fix this, we use the device's hardware randomness. The line randomSeed(analogRead(A0)); uses the unpredictable electrical "noise" from an unconnected pin (A0) to generate a unique seed every time the program starts. This makes the resulting random numbers much more varied and genuinely unpredictable.

3.3. What is a serial buffer? What is it used for? How can it be flushed? Which cases do you think you might need to apply that way?

Solution for 3.3: A serial buffer is a small, temporary storage area (usually 64 bytes) that the Arduino uses to hold incoming and outgoing data, preventing information loss during fast communication. It keeps characters ready until they are read. The best way to clear old incoming data like extra characters or leftover line endings from a previous input is to manually read and discard all available characters. This is done with the code: while (Serial.available()) Serial.read();. We need to perform this manual cleaning process before asking the user for new input. This prevents any old, unwanted characters from interfering with or corrupting the new data we are expecting.

3.4. On an Arduino, in case it gets user input with printable characters, how could you provide the board to distinguish between digits and others? **Write a program that carries this out**.

Solution for 3.4: To enable the Arduino board to distinguish between digits and other characters, the ASCII value of the received character must be checked. By implementing a conditional check, if (inputChar >= '0' && inputChar <= '9'), the program can isolate characters that are digits. This allows the code to process numbers differently than other characters like 's' or 'b'.

The program for 3.4:

```
void setup() {

 Serial.begin(9600);

 Serial.println("Enter a character (digit or letter):");

}

void loop() {

 if (Serial.available() > 0) {
```

```
  char inputChar = Serial.read();
  if (inputChar >= '0' && inputChar <= '9') {
   Serial.print("Input: '");
   Serial.print(inputChar);
   Serial.println("' is a DIGIT.");
   int digitValue = inputChar - '0';
   Serial.print("Its integer value is: ");
   Serial.println(digitValue);
  } else if (inputChar == '\n' || inputChar == '\r') {
   return;
  } else {
   Serial.print("Input: '");
   Serial.print(inputChar);
   Serial.println("' is NOT a digit.");
  }
  while (Serial.available()) Serial.read();
 }
}
```

3.5. If you are asked to create a dialog printing format to maintain communication between the user and the board, how can you align the dialogs in a way that the board's ones are on the left-hand side and the user's ones on the right-hand side of the terminal? Once the program runs, Arduino should **only ask a question and get a response** from the user. After that, the dialog should **terminate immediately**. Of course, Arduino **does not input an entry for its sayings**. You should create those dialogs yourself instead of the board, as you will do for the user's ones as well. **Write a program on Arduino to fulfill this task**. Dialogs should not be in the same line and overlap. Pay attention that, since the length of the dialogue may change, alignments are required to vary as well. E.g. ;

- Hello, I'm Arduino Uno. What is your name?.....

- Hello Arduino, my name is John.

- How are you today?.....

- I'm fine, thank you.

Solution for 3.5: To align the dialogue with the board's messages on the left and the user's responses on the right, we use a variable spacing method. The board's lines are simply printed. For the right-aligned responses, we first decide on a fixed terminal width. Then we write a function that calculates how many spaces are needed before the text to push it exactly to the right side. Since the length of each dialogue response is different, the number of spaces printed must also be different. The entire dialogue is placed inside the setup() function, and the loop() is kept empty. This ensures the communication does not overlap and terminates immediately after the conversation is complete.

The program for 3.5:

```
const int TERMINAL_WIDTH = 80;

void alignAndPrint(String text) {

  int required_spaces = TERMINAL_WIDTH - text.length();

  for (int i = 0; i < required_spaces; i++) {

    Serial.print(' ');

  }

  Serial.println(text);

}

void performDialogue(String prompt, String response) {

  Serial.println();

  Serial.println(prompt);

  alignAndPrint(response);

}

void setup() {

  Serial.begin(9600);

  while (!Serial);

  performDialogue(

    "- Hello, I'm Arduino Uno. What is your name?.....",

    "- Hello Arduino, my name is Özge."

  );

  performDialogue(

    "- How are you today?.....",

    "- I'm fine, thank you."

  );

}
```

```
void loop() {

 delay(10000);

}
```

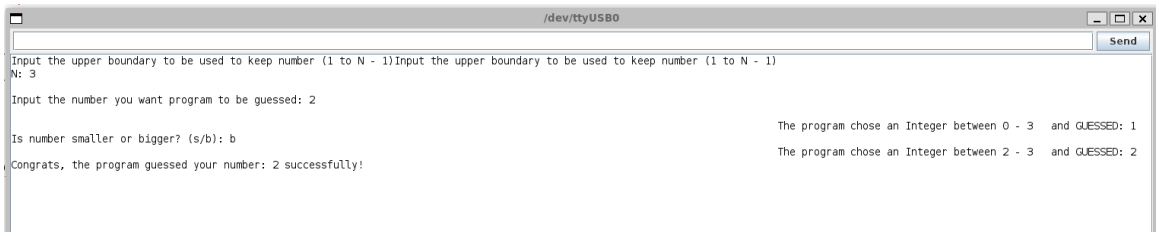
4. Labwork Evidence — Screenshots

1- Initialization and Input: The beginning of the program shows the use of input functions to set the upper boundary (N) and get the target number from the user.

2- Guesses and Alignment: The output demonstrates the right-aligned formatting protocol, where the guesses are clearly displayed on the right side of the monitor.

3- Narrowing the Range: The captured interaction validates the algorithmic integrity, as the user's response ('s' or 'b') leads to the correct and immediate narrowing of the search range (between X - Y), confirming the functionality of the guessing logic.

4- Result: The final line confirms the algorithm's success with the message.




```
unsigned long N;

unsigned long target;

unsigned long lowVal, highVal, guess;

const int Led = 7;

const int Led2 = 8;


unsigned long readUnsignedLongFromSerial(const char *prompt) {

 Serial.print(prompt);

 while (Serial.available() == 0) { delay(10); }

 long val = Serial.parseInt();

 while (Serial.available()) Serial.read();

 if (val < 0) val = 0;

 Serial.println(val);
```

```
  return (unsigned long)val;
}
char readCharFromSerial() {
  while (1) {
    while (Serial.available() == 0) { delay(10); }
    int c = Serial.read();
    if (c == -1) continue;
    if (c == '\n' || c == '\r')continue;
    while (Serial.available() && Serial.peek() != -1) {
      int p = Serial.peek();
      if (p == '\n' || p == '\r') Serial.read();
      else break;
    }
    Serial.println((char)c);
    return (char)c;
  }
}
void setup() {
  pinMode(Led, OUTPUT);
  pinMode(Led2, OUTPUT);
  Serial.begin(9600);
  while (!Serial);
  randomSeed(analogRead(A0));

  Serial.println(F("Input the upper boundary to be used to keep number (1 to N - 1)"));
  N = readUnsignedLongFromSerial("N: ");
  Serial.println();
  target = readUnsignedLongFromSerial("Input the number you want program to be guessed: ");
  if (target >= N) {
    Serial.println(F("Error: target must be less than N."));
    while (1) { delay(1000); }
```

```
  }
  lowVal = 0;
  highVal = N;
  Serial.println();
  delay(200);
}
void loop() {
  if (lowVal > highVal) {
    Serial.println(F("Inconsistent state: low > high. Exiting."));
    while (1) { delay(1000); }
  }
  unsigned long maxArg = (highVal == 0xFFFFFFFFUL) ? 0xFFFFFFFFUL : highVal + 1;
  if (maxArg <= lowVal)
    guess = lowVal;
  } else {
    guess = random(lowVal, maxArg);
  }
  Serial.print(F("                                                                The program chose an Integer between "));
  Serial.print(lowVal);
  Serial.print(F(" - "));
  Serial.print(highVal);
  Serial.print(F("   and GUESSED: "));
  Serial.println(guess);

  if (guess == target) {
    int i = 0;
    digitalWrite(Led, HIGH);
    delay(300);
    digitalWrite(Led, LOW);
    delay(300);
    Serial.print(F("Congrats, the program guessed your number: "));
```

```
  Serial.print(guess);
  Serial.println(F(" successfully!"));
  while (1) { delay(1000); }
 }
 else {
  int i = 0;
  digitalWrite(Led2, HIGH);
  delay(300);
  digitalWrite(Led2, LOW);
  delay(300);
 Serial.print(F("Is number smaller or bigger? (s/b): "));
 char resp = readCharFromSerial();
 if (resp == 'b' || resp == 'B') {
  if (guess == 0xFFFFFFFFUL) {
    Serial.println(F("Overflow risk when setting low = guess + 1. Exiting."));
    while (1) { delay(1000); }
  }
  lowVal = guess + 1;
 } else if (resp == 's' || resp == 'S') {
  if (guess == 0) {
    Serial.println(F("Underflow risk when setting high = guess - 1. Exiting."));
    while (1) { delay(1000); }
  }
  highVal = (guess == 0) ? 0 : guess - 1;
 } else {
  Serial.println(F("Invalid response. Use 's' or 'b'. Halting."));
  while (1) { delay(1000); }
 }
 delay(250);
 }
 }
```